

DANKE MOTORSPORT

Changelog — Alterações desde o último review

Commits analisados: 4903632 (Responsividade) → 30dfec4 (keep-alive fix)
Período: 02/06/2026 a 06/06/2026 | Autores: Johan, Gustavo, David

Hash	Data	Autor	Mensagem
30dfec4	06/06	David	fix(ci): parse C#-formatted connection string in keep-alive
d5510dd	06/06	David	feat(auth): JWT authentication + route protection + keep-alive workflow
1d7bf93	06/06	Gustavo	feat(api): RevisoesController — CRUD completo
6451bdd	06/06	Gustavo	feat(api): GET por id, PUT e DELETE em Clientes e Funcionarios
4903632	02/06	Johan	Responsividade — media queries na LandingPage e Navbar

1. Backend — C# .NET

1.1 AuthController.cs — NOVO (David)

Arquivo criado do zero. Implementa autenticação JWT para os dois perfis de usuário.

- **POST /danke/auth/login** — busca o e-mail primeiro em Clientes, depois em Funcionarios. Valida a senha com `BCrypt.Verify()`. Gera um token JWT com claims de Id, Role, Nome e Email. Retorna o token e um objeto user simplificado (id, nome, email, role).
- Token com expiração de **7 dias**. Algoritmo `HmacSha256`.

■■ Chave JWT hardcoded como fallback no código:
"danke_motorsport_super_secret_key_1234567890_default". Se `Jwt:Key` não estiver no `appsettings`, esse valor é usado silenciosamente. Risco: qualquer pessoa que leia o repositório consegue forjar tokens válidos.

1.2 Program.cs — ATUALIZADO (David)

- JWT Bearer Authentication configurado com `AddAuthentication / AddJwtBearer`.
- **CORS** habilitado para `http://localhost:5173` (porta do Vite). Faltará atualizar para a URL do Vercel no deploy.
- `app.UseAuthentication()` e `app.UseAuthorization()` adicionados na pipeline — correto e necessário.

1.3 ClientesController.cs — ATUALIZADO (Gustavo)

- POST agora faz `BCrypt.HashPassword()` antes de salvar e limpa o campo Senha no retorno. Correto.

- Adicionados: GET /{id}, PUT /{id}, DELETE /{id} com validação de ModelState e NotFound.

■ Nenhum dos endpoints tem [Authorize]. PUT /{id} não verifica se o cliente logado é o dono do registro que está editando — qualquer usuário não autenticado pode alterar ou deletar qualquer cliente.

■ GET /danke/clientes retorna a lista completa incluindo o campo SenhaHash de todos os clientes. Necessário usar DTOs para não expor dados sensíveis.

1.4 FuncionariosController.cs — ATUALIZADO (Gustavo)

- Adicionados: GET /{id}, PUT /{id}, DELETE /{id}. Mesma estrutura do ClientesController.

■ Mesmos problemas de segurança: sem [Authorize] em nenhum endpoint.

1.5 RevisaoController.cs — IMPLEMENTADO (Gustavo)

Controller que estava vazio foi completamente implementado. Detalhe dos endpoints:

- **GET /danke/revisao** [Authorize] — retorna revisões filtrando por role: Cliente vê apenas as suas, Funcionario vê apenas as atribuídas a ele.
- **GET /danke/revisao/cliente** [Authorize(Roles="Cliente")] — revisões do cliente autenticado, ordenadas por data.
- **GET /danke/revisao/funcionario** [Authorize(Roles="Funcionario")] — revisões do funcionário, com Include do Cliente.
- **GET /danke/revisao/pendentes** [Authorize(Roles="Funcionario")] — todas com status 'Pendente', com Include do Cliente.
- **GET /danke/revisao/{id}** [Authorize] — verifica ownership: cliente só acessa a própria revisão.
- **POST /danke/revisao** [Authorize(Roles="Cliente")] — ClientId extraído do JWT claim (correto, não do body). Status inicial: 'Pendente'. IdFuncionario: null.
- **PATCH /danke/revisao/{id}** [Authorize(Roles="Funcionario")] — atualiza status. Na primeira interação (IdFuncionario == null), atribui o funcionário logado. Se status == 'Concluído', define DatFinalizacao = DateTime.UtcNow.
- **DELETE /danke/revisao/{id}** [Authorize] — cliente só deleta a própria revisão.

■ Lógica de negócio bem implementada. IdCliente vindo do JWT é a abordagem correta e segura.

1.6 Models — ATUALIZADOS (David/Gustavo)

- **Cliente.cs**: campo **Senha** (string, Required, column 'senha') adicionado.
- **Funcionario.cs**: campos **Email** e **Senha** adicionados.
- **Revisao.cs**: **IdFuncionario** alterado de int para int? (nullable) — correto, revisão recém-criada não tem funcionário.

1.7 Migration AddAuthAndNullableEmployee (David)

- Migration gerada e aplicada. Adiciona colunas 'senha' e 'email' nas tabelas e torna id_funcionario nullable em revisoes.

1.8 GitHub Actions — keep-alive.yml (David)

- Workflow criado em `.github/workflows/keep-alive.yml`.
- Agenda: cron a cada 5 dias às 00:00 UTC + disparo manual pelo painel do GitHub.
- Função: conecta ao PostgreSQL do Supabase via psql e executa `SELECT 1` para evitar que o projeto gratuito seja pausado por inatividade.
- Credenciais lidas de `secrets.SUPABASE_CONNECTION_STRING` — correto, não exposto no código.

■ Solução criativa e necessária para garantir disponibilidade 24/7 no plano gratuito do Supabase.

2. Frontend — React

2.1 AuthContext.jsx — NOVO (David)

- Context criado com `useState` para `user` e `loading`. Lê `localStorage` na inicialização (`useEffect`).
- `login()`: faz `POST /danke/auth/login` via `axios`, salva `token` e `user` no `localStorage`.
- `logout()`: limpa `localStorage` e reseta o estado.
- Hook `useAuth()` exportado com validação — lança erro se usado fora do `AuthProvider`.

■■ Token JWT armazenado em `localStorage`. Em aplicações com dados sensíveis, `httpOnly` cookies são mais seguros (resistentes a XSS). Para o escopo do projeto, `localStorage` é aceitável.

2.2 ProtectedRoute.jsx — NOVO (David)

- Componente que lê `signed`, `user` e `loading` do `AuthContext`.
- Exibe tela de carregando enquanto `loading=true` (evita flash de redirecionamento).
- Redireciona para `/auth` se não autenticado.
- Suporta `allowedRoles`: redireciona para `/` se a role do usuário não está na lista.

■ Implementação de guard de rota robusta com suporte a roles — cobre o critério de acesso apenas após autenticação.

2.3 routes/index.jsx — ATUALIZADO (David)

- `ClientDashboard` protegido por `ProtectedRoute` com `allowedRoles=['Cliente']`.
- `EmployeeDashboard` protegido com `allowedRoles=['Funcionario']`.
- Estrutura usa `Outlet` — padrão correto do `react-router-dom v6+`.

2.4 main.jsx — ATUALIZADO (David)

- `RouterProvider` conectado corretamente — roteamento funcionando.
- `AuthProvider` envolve o `RouterProvider` — Context disponível em toda a árvore.

■■ Imports não utilizados: `Route` e `Router` importados do `react-router-dom` mas não usados. Ruído no código, mas não causa erro.

2.5 Auth.jsx — ATUALIZADO (David)

- `onSubmit` adicionado em ambos os formulários (login e cadastro).

- Login: chama login() do AuthContext e redireciona por role (Cliente→/client-dashboard, Funcionario→/employee-dashboard).
- Cadastro: valida senhas iguais, faz POST /danke/clientes, exibe alerta e volta para login.
- Estados loading e error adicionados. Botão desabilitado durante loading.
- Mensagem de erro exibida inline abaixo dos campos.

2.6 ClientDashboard.jsx — NOVO (David)

- Tela do cliente após login. Header com nome do usuário e botão de logout.
- Formulário de solicitação: seleção de plano (Bronze/Silver/Gold via radio cards) + datetime-local.
- POST /danke/revisao com dados do formulário. Recarrega histórico após sucesso.
- Tabela de histórico com GET /danke/revisao/cliente — exibe id, plano, data, status e finalização.
- Status com badge colorido (Pendente/Em Andamento/Concluído).

2.7 EmployeeDashboard.jsx — NOVO (David)

- Kanban com 3 colunas: Pendentes (GET /pendentes), Em Andamento e Concluídas (GET /funcionario).
- 'Iniciar' faz PATCH /{id} com status 'Em Andamento'. 'Concluir' faz PATCH com 'Concluído'.
- Cards exibem dados do cliente (nome, telefone via .cliente) e data de agendamento.
- Após cada ação, recarrega os dados do servidor.

2.8 services/api.js — NOVO (David)

- Instância axios com baseURL lida de VITE_API_URL ou fallback para localhost:5136.
- Interceptor de request injeta o Bearer token do localStorage em todos os requests.

■■ Porta de fallback localhost:5136 diverge da porta documentada no setup.xml (localhost:5000). Verificar qual porta o dotnet run realmente usa e padronizar.

2.9 Responsividade — NOVO (Johan, 02/06)

- LandingPage.css: @media (max-width: 768px) — background reposicionado, texto e botões adaptados para coluna.
- Navbar.css: @media (max-width: 768px) — layout ajustado para mobile.

■■ Auth.jsx, ClientDashboard e EmployeeDashboard ainda não possuem media queries. Inputs com largura fixa (25vw) quebram em mobile.

2.10 dist/ commitado — PROBLEMA

■ A pasta frontend/dist/ (build de produção) foi commitada no repositório. Artefatos de build não pertencem ao git — aumentam o tamanho do repo e causam conflitos desnecessários. Adicione frontend/dist/ ao .gitignore e rode git rm --cached -r frontend/dist/.